

Competitive Programming

Section 30 Editorial: GCD, LCM, and Diophantine Equations

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
2	A. GCD LCM Queries	4
2.1	Restatement	4
2.2	Algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	5
3	B. General Inverse	6
3.1	Restatement	6
3.2	Algorithm	6
3.3	Correctness proof	6
3.4	Complexity	6
3.5	Full C++ solution	6
3.6	Full Python solution	7
4	C. Linear Diophantine	9
4.1	Restatement	9
4.2	Algorithm	9
4.3	Correctness proof	9
4.4	Complexity	9
4.5	Full C++ solution	9
4.6	Full Python solution	10
5	Testing discussion	12
6	What to carry forward	13

1 Editorial overview

Problem	Tool	Output
A. GCD LCM Queries	Euclid and lcm formula	<code>gcd lcm</code>
B. General Inverse	extended Euclid	inverse or <code>-1</code>
C. Linear Diophantine	Bezout scaling	one solution or impossible

2 A. GCD LCM Queries

2.1 Restatement

For each pair a, b , print their greatest common divisor and least common multiple.

2.2 Algorithm

```
g = gcd(a,b)
l = a / g * b
```

2.3 Correctness proof

Euclid's algorithm computes the greatest common divisor because replacing (a, b) by $(b, a \bmod b)$ preserves the set of common divisors. When the second number becomes zero, the first is the greatest common divisor.

The formula $\text{gcd}(a, b) * \text{lcm}(a, b) = a * b$ is standard from prime exponents: for each prime, gcd takes the minimum exponent and lcm takes the maximum; their sum equals the sum of exponents in $a * b$. Therefore $a / \text{gcd}(a, b) * b$ is the lcm. $\square$

2.4 Complexity

```
Time:  $O(q \log \min(a, b))$ 
Memory:  $O(1)$ 
```

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
```

```

if (!(cin >> q)) return 0;
while (q--) {
    long long a, b;
    cin >> a >> b;
    long long g = gcd(a, b);
    cout << g << ' ' << a / g * b << '\n';
}
return 0;
}

```

2.6 Full Python solution

```

import math
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    idx = 1
    out = []
    for _ in range(q):
        a, b = data[idx], data[idx + 1]
        idx += 2
        g = math.gcd(a, b)
        out.append(f"{g} {a // g * b}")
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

3 B. General Inverse

3.1 Restatement

For each a, m , find the smallest non-negative x with $a \cdot x \equiv 1 \pmod{m}$, or print -1 .

3.2 Algorithm

Run extended Euclid:

$$a \cdot x + m \cdot y = \gcd(a, m)$$

If the gcd is not 1, no inverse exists. Otherwise, $a \cdot x + m \cdot y = 1$, so $a \cdot x \equiv 1 \pmod{m}$. Normalize x into $[0, m-1]$.

When $m = 1$, the only residue is 0, so the local problem prints 0.

3.3 Correctness proof

An inverse exists iff there is an integer x with $a \cdot x - 1$ divisible by m , equivalently $a \cdot x + m \cdot y = 1$ for some integer y . By Bezout's identity, the set of all integer combinations $a \cdot x + m \cdot y$ is exactly the multiples of $\gcd(a, m)$. Therefore 1 is representable iff $\gcd(a, m) = 1$. Extended Euclid constructs such an x when it exists. $\square$

3.4 Complexity

Time: $O(\log \min(a, m))$
Memory: $O(1)$

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

long long egcd(long long a, long long b, long long &x, long long &y) {
    if (b == 0) {
        x = 1;
    }
```

```

        y = 0;
        return a;
    }
    long long x1, y1;
    long long g = egcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    while (q--) {
        long long a, m;
        cin >> a >> m;
        if (m == 1) {
            cout << 0 << '\n';
            continue;
        }
        long long x, y;
        long long g = egcd(a, m, x, y);
        if (g != 1) {
            cout << -1 << '\n';
        } else {
            x %= m;
            if (x < 0) x += m;
            cout << x << '\n';
        }
    }
    return 0;
}

```

3.6 Full Python solution

```

import sys

def egcd(a: int, b: int) -> tuple[int, int, int]:
    if b == 0:
        return a, 1, 0
    g, x1, y1 = egcd(b, a % b)
    return g, y1, x1 - (a // b) * y1

```

```

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    idx = 1
    out = []
    for _ in range(q):
        a, m = data[idx], data[idx + 1]
        idx += 2
        if m == 1:
            out.append("0")
            continue
        g, x, _ = egcd(a, m)
        out.append(str(x % m) if g == 1 else "-1")
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

4 C. Linear Diophantine

4.1 Restatement

For each a, b, c , find any integer pair x, y satisfying $a \cdot x + b \cdot y = c$, or report impossible.

4.2 Algorithm

```
g, x0, y0 = extended_gcd(a,b)
if c % g != 0:
    IMPOSSIBLE
else:
    scale = c / g
    x = x0 * scale
    y = y0 * scale
```

4.3 Correctness proof

Extended Euclid gives $a \cdot x_0 + b \cdot y_0 = g$, where $g = \gcd(a, b)$. If c is not a multiple of g , no integer combination of a and b can equal c , because every such combination is divisible by g .

If $c = g \cdot \text{scale}$, multiplying the Bezout identity by scale gives:

$$a \cdot (x_0 \cdot \text{scale}) + b \cdot (y_0 \cdot \text{scale}) = c$$

so the algorithm prints a valid solution. $\square$

4.4 Complexity

```
Time:  $O(q \log \min(a, b))$ 
Memory:  $O(1)$ 
```

4.5 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

long long egcd(long long a, long long b, long long &x, long long &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    long long x1, y1;
    long long g = egcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    while (q--) {
        long long a, b, c;
        cin >> a >> b >> c;
        long long x, y;
        long long g = egcd(a, b, x, y);
        if (c % g != 0) {
            cout << "IMPOSSIBLE\n";
        } else {
            long long scale = c / g;
            cout << x * scale << ' ' << y * scale << '\n';
        }
    }
    return 0;
}

```

4.6 Full Python solution

```

import sys

def egcd(a: int, b: int) -> tuple[int, int, int]:
    if b == 0:
        return a, 1, 0
    g, x1, y1 = egcd(b, a % b)
    return g, y1, x1 - (a // b) * y1

```

```

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    idx = 1
    out = []
    for _ in range(q):
        a, b, c = data[idx], data[idx + 1], data[idx + 2]
        idx += 3
        g, x, y = egcd(a, b)
        if c % g != 0:
            out.append("IMPOSSIBLE")
        else:
            scale = c // g
            out.append(f"{x * scale} {y * scale}")
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

5 Testing discussion

The local checker compares against the reference construction, so fixed sample outputs are one valid solution, not the only possible solution. On real judges for Diophantine problems, either the output checker accepts any valid pair or the problem asks for a specific normalized solution.

Important tests:

```
coprime a,b with c=1  
gcd(a,b) not dividing c  
m=1 for modular inverse  
large coprime values
```

6 What to carry forward

GCD tells you what is divisible, extended Euclid tells you how to build the linear combination, and modular inverse is just a special case of that linear combination.